

CETAM / INSTITUTO BENJAMIN CONSTANT (IBC)

PROJETO 2: LOJA VIRTUAL DE ROUPAS — LINDA MODA

Sistema de Controle de Clientes e Pedidos

Curso Técnico em Informática — Projeto de Sistemas Computacionais Web



INSTRUTOR(A)

Prof. Cleverson Silva Sobrinho

EQUIPE (GRUPO 04)

Isaque Reis, Joquebede Barboza, Suelen Tavares, Suely Nascimento e Sophia Roque Martins

SUMÁRIO

1

Contextualização — O problema comercial e a solução automatizada

2

Apresentação da Arquitetura — Organização em camadas com Spring Boot

3

Demonstração Funcional — Fluxo prático da aplicação Linda Moda

4

Demonstração Técnica — Doutrina de código e Spring Data JPA

5

Banco de Dados — Modelo relacional e cardinalidades (loja_virtual_roupas)

6

Considerações Finais — Conclusão do sistema e espaço para arguição

O Desafio do Varejo de Moda

O Problema

A gestão manual de vendas de vestuário e o uso de planilhas descentralizadas geram sobrecarga, erros no controle de estoque e falta de visibilidade das operações comerciais.

A Solução

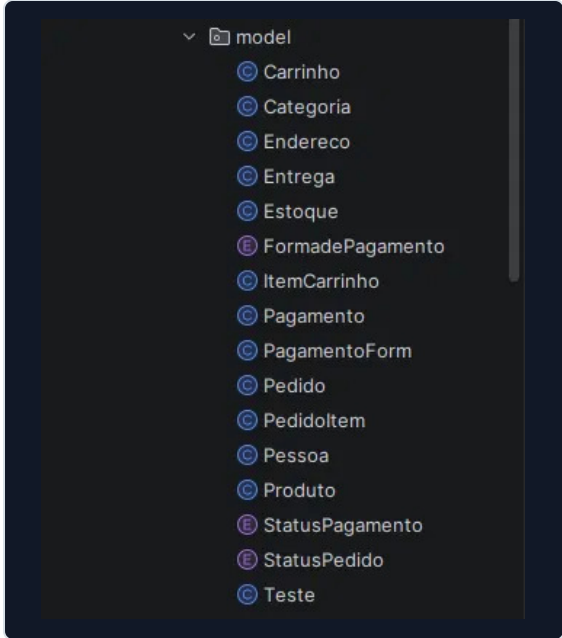
Desenvolvimento da plataforma web Linda Moda para automatizar o cadastro unificado de pessoas, gerenciar o catálogo de produtos, controlar o carrinho de compras, processar pedidos e pagamentos, e monitorar a logística de entrega.

Apresentação da Arquitetura (Spring Boot)

A arquitetura do projeto é dividida em camadas, separando as responsabilidades de cada parte.

Model

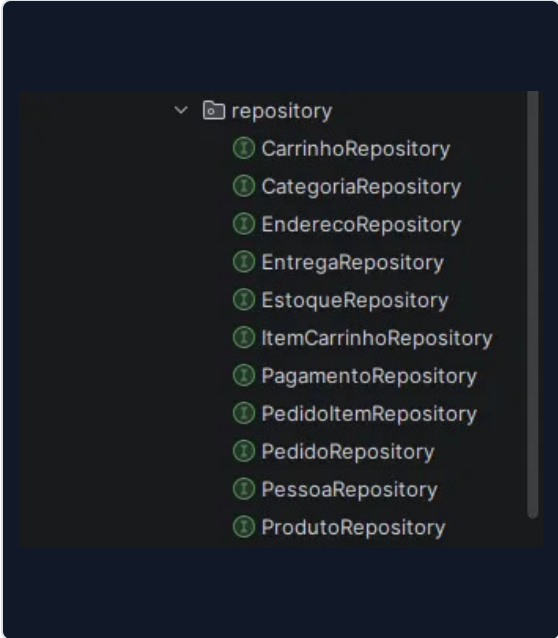
Representa os dados da aplicação. A classe Pessoa faz o mapeamento da tabela pessoa no banco de dados.



```
model
├── Carrinho
├── Categoria
├── Endereco
├── Entrega
├── Estoque
├── FormadePagamento
├── ItemCarrinho
├── Pagamento
├── PagamentoForm
├── Pedido
├── PedidoItem
├── Pessoa
├── Produto
├── StatusPagamento
├── StatusPedido
└── Teste
```

Repository

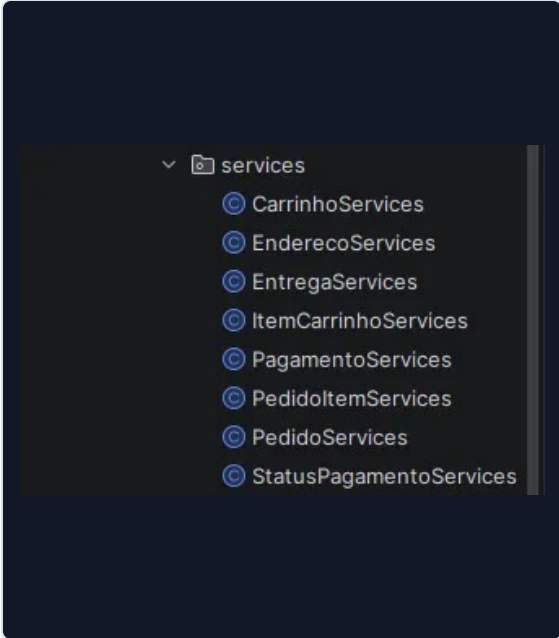
Realiza a comunicação com o banco de dados. O PessoaRepository utiliza JpaRepository para permitir operações de persistência, consulta, alteração e exclusão.



```
repository
├── CarrinhoRepository
├── CategoriaRepository
├── EnderecoRepository
├── EntregaRepository
├── EstoqueRepository
├── ItemCarrinhoRepository
├── PagamentoRepository
├── PedidoItemRepository
├── PedidoRepository
├── PessoaRepository
└── ProdutoRepository
```

Service

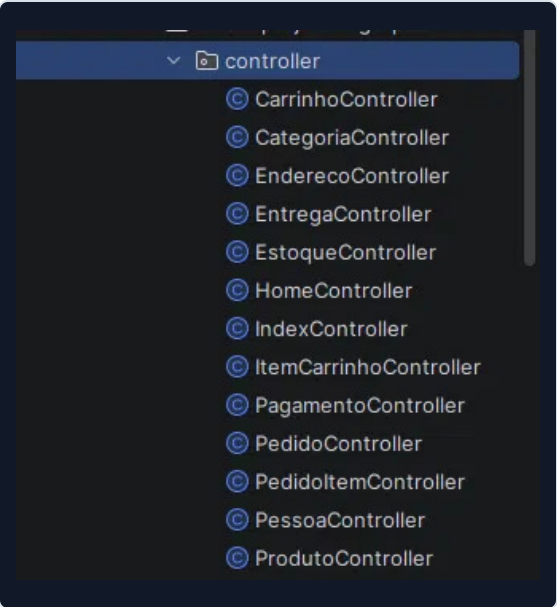
Concentra as regras de negócio. O PessoaService organiza operações de cadastro, consulta, atualização, exclusão, validação e diferenciação dos tipos de pessoa.



```
services
├── CarrinhoServices
├── EnderecoServices
├── EntregaServices
├── ItemCarrinhoServices
├── PagamentoServices
├── PedidoItemServices
├── PedidoServices
└── StatusPagamentoServices
```

Controller

Recebe as requisições da aplicação e direciona as operações. O PessoaController realiza operações de listar, consultar, cadastrar, atualizar e excluir pessoas, além de trabalhar com Cliente, Fornecedor e Funcionário.



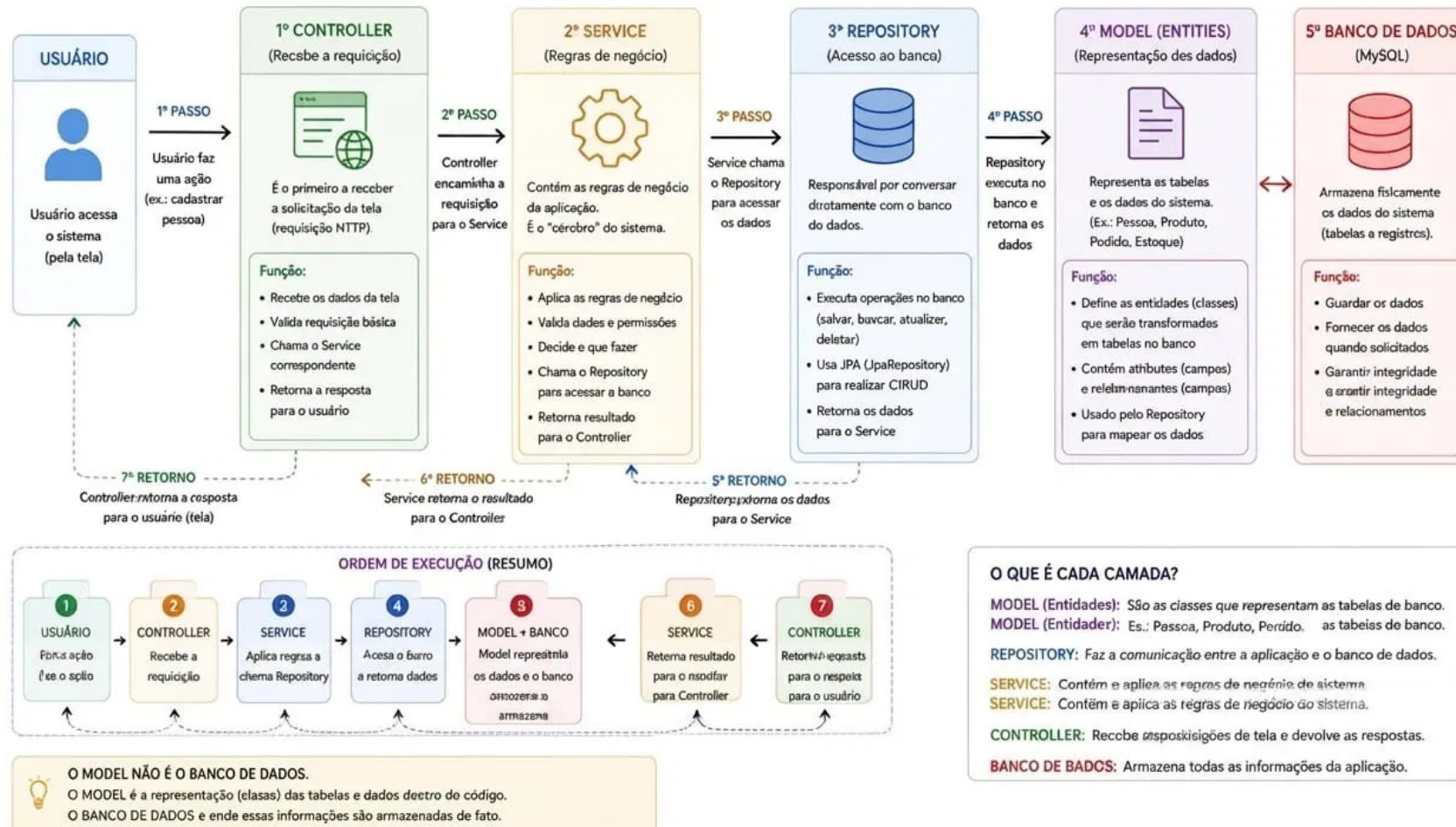
```
controller
├── CarrinhoController
├── CategoriaController
├── EnderecoController
├── EntregaController
├── EstoqueController
├── HomeController
├── IndexController
├── ItemCarrinhoController
├── PagamentoController
├── PedidoController
├── PedidoItemController
├── PessoaController
└── ProdutoController
```

Ordem de Execução e Função de Cada Camada

Fluxo de comunicação entre usuário, Controller, Service, Repository, Model e Banco de Dados.

ARQUITETURA DA APLICAÇÃO – LINDA MODA

Ordem de Execução e Função de Cada Camada (Spring Boot)



Fluxo Prático - Aplicação Linda Moda

1

Abrir o sistema e acessar o cadastro.

2

Informar os dados da pessoa.

3

Definir o tipo: Cliente, Fornecedor ou Funcionário.

4

Salvar o cadastro.

5

Consultar o registro cadastrado.

6

Demonstrar a diferenciação pelo tipo.

7

Alterar alguma informação e salvar.

8

Excluir um registro e mostrar que não aparece mais.

4. DEMONSTRAÇÃO TÉCNICA

Código Spring Boot em Destaque

Anotação @Entity

Informa ao JPA que a classe Pessoa representa uma entidade que será persistida no banco de dados.

```
package cetam.projeto02grupo04.model;

import ...

@Entity @ zack-reis +1
@Table(name = "pessoa")
public class Pessoa {
```

PessoaRepository

Utiliza JpaRepository e disponibiliza operações de persistência (salvar, consultar, alterar, excluir) sem implementação manual.

```
public interface PessoaRepository 1 usage @ zack-reis *
    extends JpaRepository<Pessoa, Long> {
    List<Pessoa> findByTipoIgnoreCase(String tipo);
}
```

Controller e Service

O Controller recebe requisições e o Service concentra as regras de negócio. Essa separação mantém cada camada com sua responsabilidade.

CONTROLLER - PessoaController.java

```
01 package cetam.projeto02grupo04.controller;
02
03 @RestController
04 @RequestMapping("/pessoa")
05 public class PessoaController {
06     @GetMapping("/tipo/{tipoPessoa}")
07     public List<Pessoa> buscarPorTipo(..)
08     {
09         return services.buscarPorTipo(tipo);
10     }
11     @PostMapping
12     public Pessoa salvar(@RequestBody Pessoa pessoa) {
13         return services.salvar(pessoa);
14     }
15 }
```


Banco de Dados (loja_virtual_roupas)

MySQL e Modelagem

Modelagem de Dados

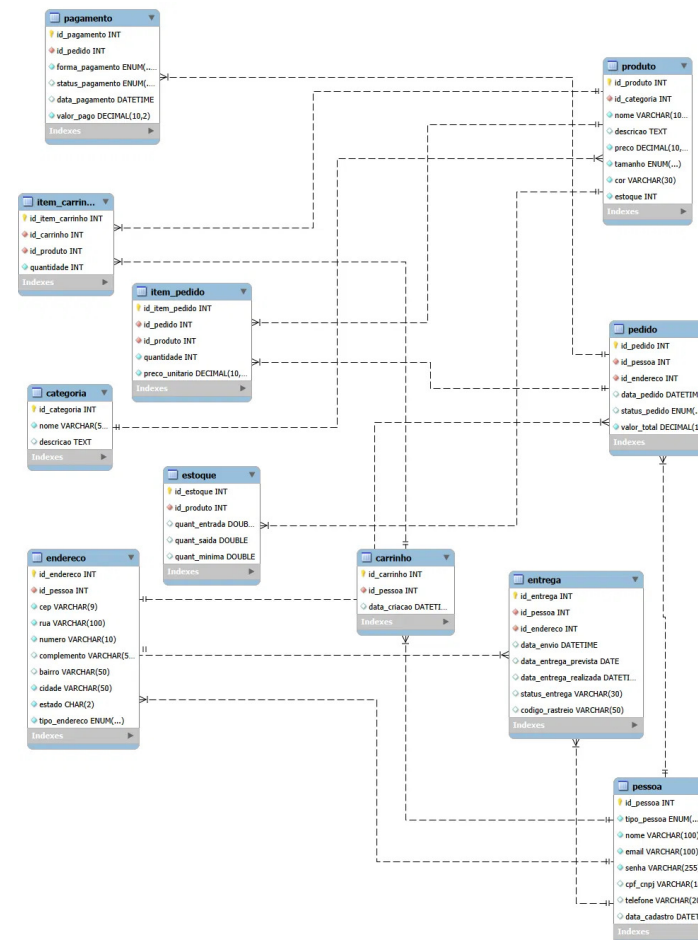
Realizada no MySQL Workbench, com definição de atributos, chaves primárias, chaves estrangeiras, DER e scripts SQL.

Tabela Pessoa

Utilizada pelo backend e mapeada pela classe Pessoa.

Diferenciação por Tipo

O atributo tipo permite diferenciar Cliente, Fornecedor e Funcionário. Essa diferenciação é utilizada no Repository, Controller e Service.



Conclusão

O sistema entregue cumpre o objetivo de facilitar o gerenciamento de clientes, fornecedores, funcionários e pedidos da loja Linda Moda, organizando os dados de forma eficiente com Java, Spring Boot e MySQL.

1

Gestão Centralizada

Cadastro e controle de todos os perfis da loja (Cliente, Fornecedor, Funcionário) em um único sistema.

2

Arquitetura Limpa

Separação clara de responsabilidades entre Model, Repository, Service e Controller.

3

Banco de Dados Seguro

Persistência relacional no MySQL com mapeamento eficiente via JPA e Hibernate.

Espaço aberto para a arguição e considerações da banca avaliadora.

Obrigado pela atenção! 🙏